Name: Arjun B R

Intern ID: IN126049802

Title: RAG-Based Customer Support Assistant

LOW-LEVEL DESIGN (LLD)

Objective

The objective of this Low-Level Design is to describe how the RAG-based Customer Support Assistant is implemented internally. It explains the system modules, data structures, workflow execution, decision logic, and error handling in detail.

1. Module-Level Design

The system is divided into multiple modules, each responsible for a specific task.

- **Document Processing Module**

  This module handles the ingestion of PDF files. It uses a PDF loader to extract text content from the uploaded document. The extracted data is then passed to the chunking module for further processing.

- **Chunking Module**

  The chunking module splits the extracted text into smaller overlapping segments. This improves retrieval accuracy and ensures that important information is not lost between chunks.

  - Chunk size: ~700 characters
  - Overlap: ~100 characters

- **Embedding Module**

  This module converts each text chunk into a vector representation using a pre-trained embedding model (MiniLM). These embeddings capture semantic meaning, allowing similarity-based search.

- **Vector Storage Module**

  All generated embeddings are stored in ChromaDB. This enables efficient indexing and fast retrieval of relevant chunks based on similarity search.

- **Retrieval Module**

  This module retrieves the most relevant chunks based on the user query. It uses Max Marginal Relevance (MMR) to ensure diversity and reduce redundancy in results.

- **Query Processing Module**

  This module combines retrieved chunks into a context and prepares a prompt for the LLM. It ensures that the answer is generated strictly from the provided context.

- **Graph Execution Module**

  LangGraph is used to manage the workflow execution. It defines nodes and transitions, ensuring proper control flow between processing, decision-making, and output generation.

- **HITL Module**

  This module handles cases where the system cannot generate a reliable answer. It prompts the user for manual input and integrates the human response into the system output.

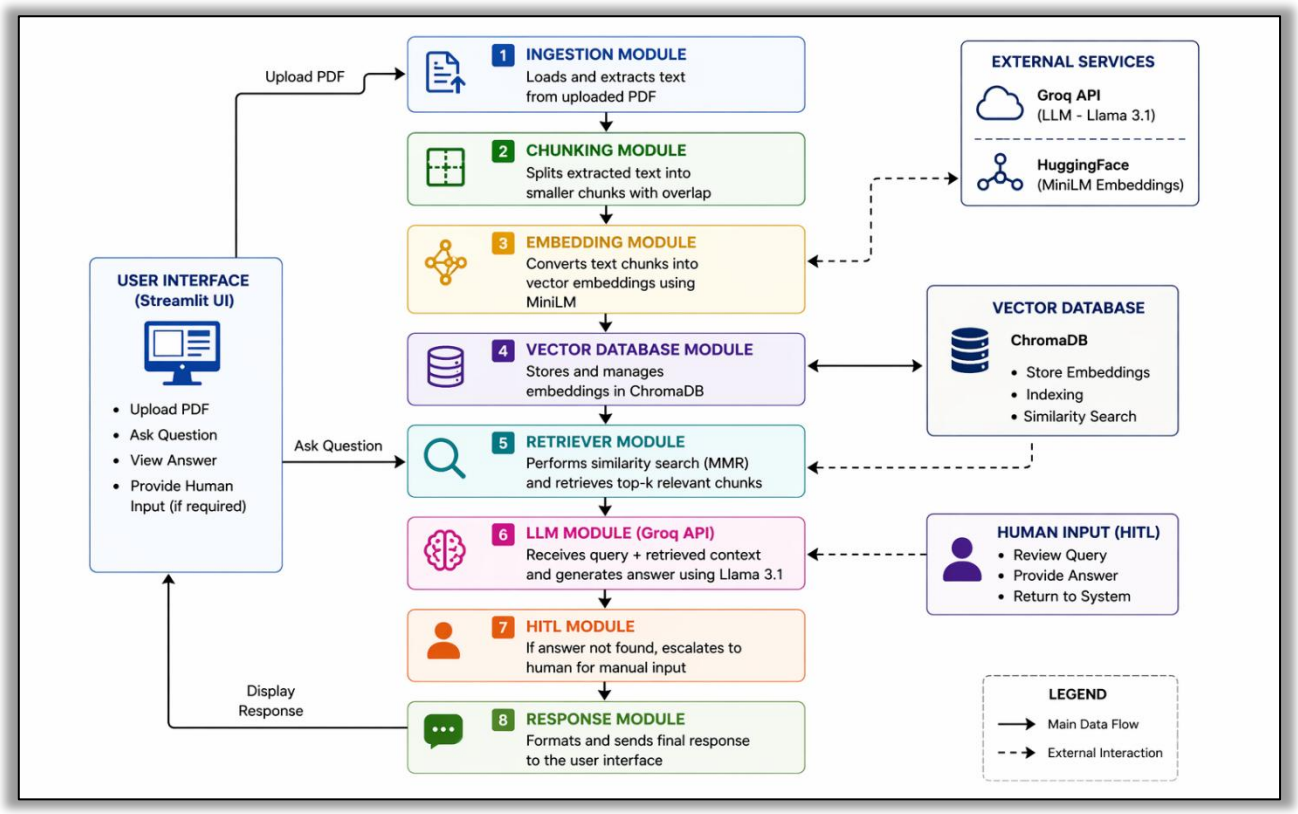

Fig: Module Interaction Diagram

2. Data Structures

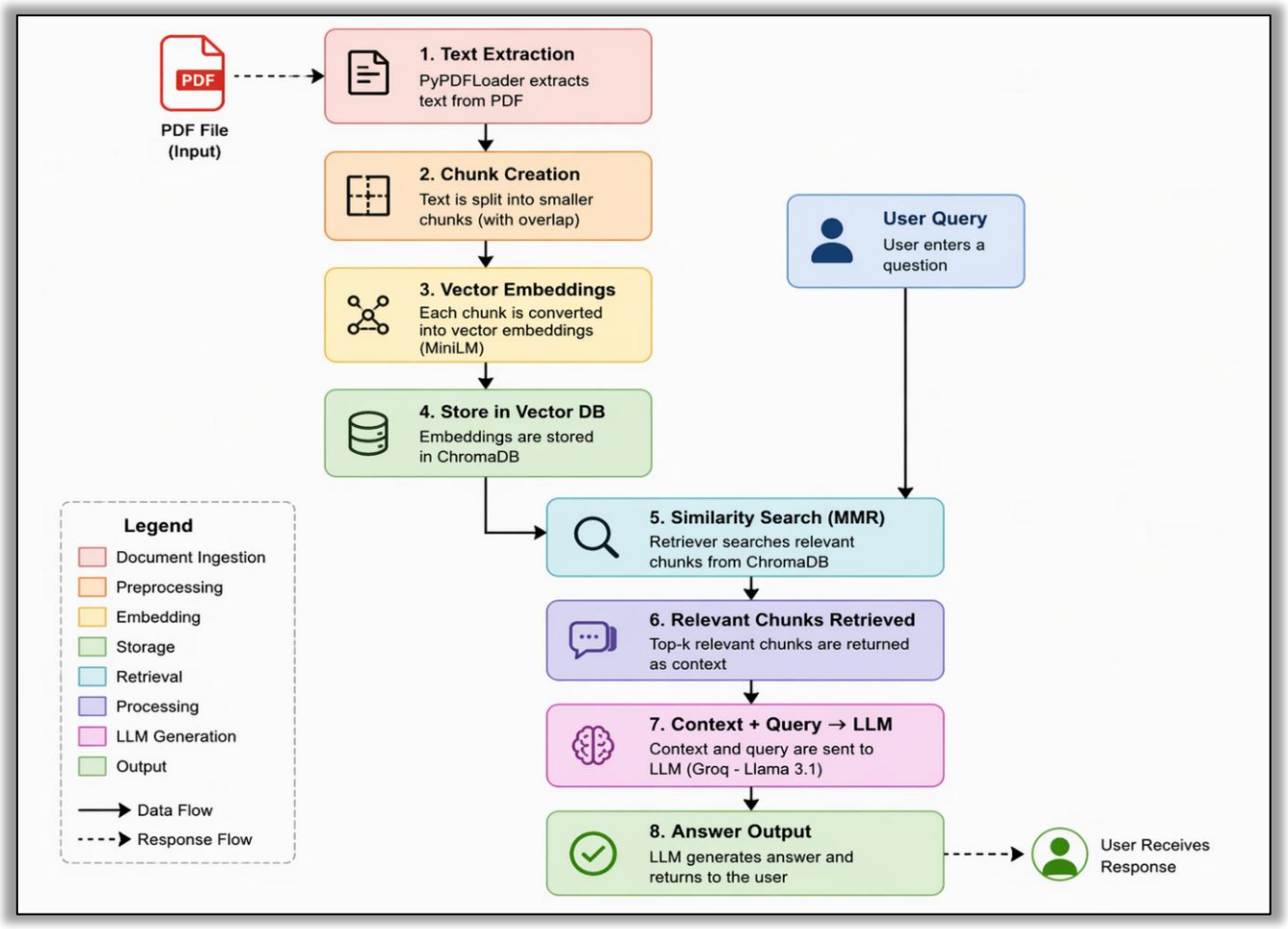

Fig: Data Flow Diagram

Document Representation:

{

content: "full text of PDF",

metadata: { source, page_number }

}

Chunk Format:

{

chunk_id: int,

text: "chunk content",

metadata: { page_number }

}

Embedding Structure

```
[0.12, -0.45, 0.78, ...]
```

Each chunk is represented as a high-dimensional vector.

Query-Response Schema

```
{
  query: "user question",
  context: "retrieved chunks",
  answer: "generated output"
}
```

Graph State Object

```
{
  query: str,
  context: str,
  answer: str
}
```

This state is passed between nodes in LangGraph.

3. Workflow Design (LangGraph)

- **Nodes**

  Processing Node

  - Retrieves relevant chunks
  - Generates answer using LLM

  Output Node

  - Returns final answer or triggers HITL

- **Edges**

START ⟶ PROCESS ⟶

⟶ OUTPUT (if answer found)

⟶ HITL (if not found) → OUTPUT

- **State Flow**

The state object carries:

- Query
- Retrieved context
- Generated answer

This state is updated at each node and passed forward.

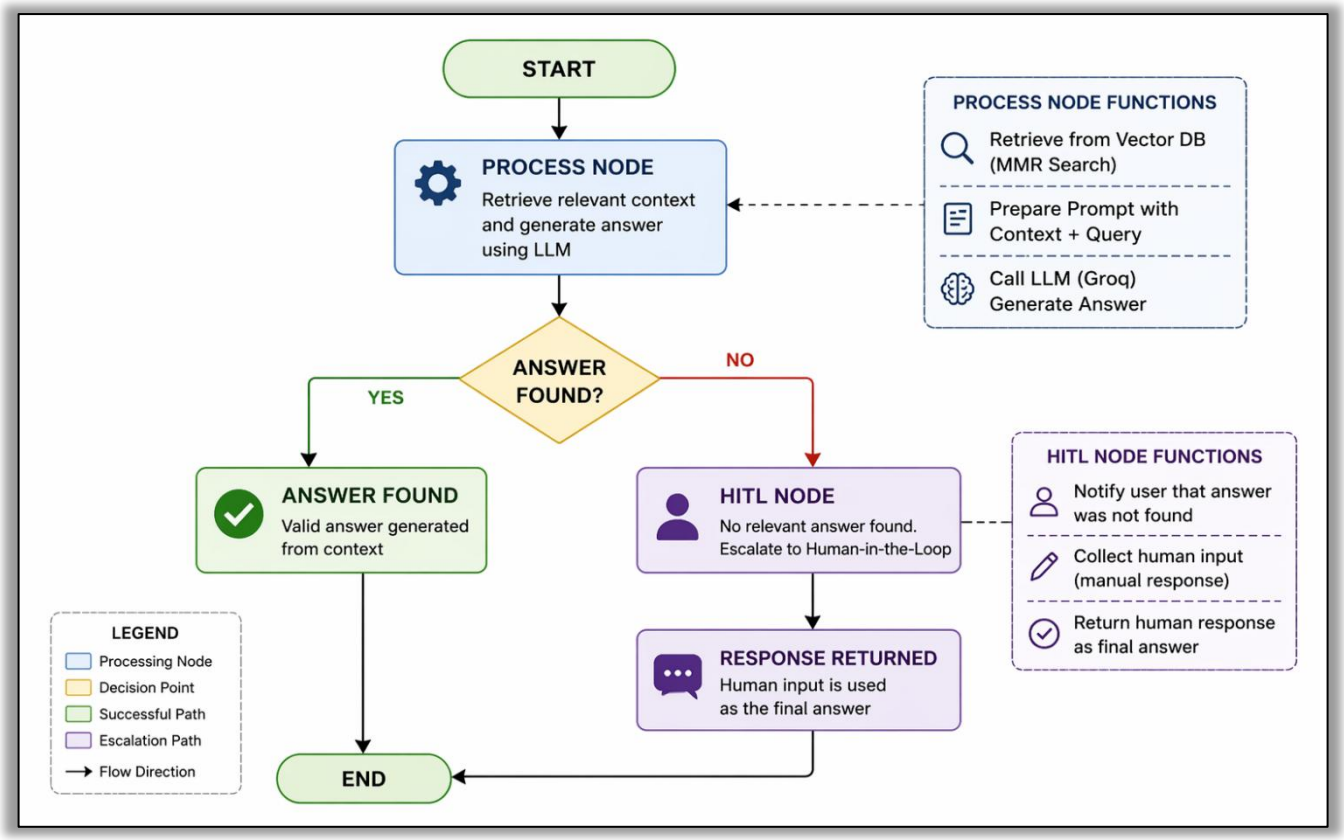

Fig: LangGraph Workflow Diagram

4. Conditional Routing Logic

The system uses conditional logic to decide whether to answer or escalate.

➢ **Answer Generation Criteria**

- Relevant chunks are successfully retrieved
- Context contains sufficient information
- LLM generates a meaningful response

➢ **Escalation Criteria**

HITL is triggered when:

- No relevant chunks are found
- Context does not contain the answer
- Query is too complex or ambiguous
- LLM returns “NOT FOUND”

➢ **Low Confidence Handling**

If the system is unsure about the answer due to weak context or unclear retrieval, it avoids guessing and triggers HITL.

➢ **Missing Context**

If retrieved chunks do not contain useful information, the system does not generate an answer and escalates.

➢ **Complex Query Handling**

For multi-part or unclear queries, the system may fail to retrieve sufficient context, leading to escalation.

5. HITL Design

- **When Escalation is Triggered**
  - When answer = “NOT FOUND”
  - When retrieval fails
  - When confidence is low
- **What Happens After Escalation**
  - System displays: “Human input required”
  - User enters manual response
- **Integration of Human Response**
  - Human input is treated as final answer
  - Displayed in chat interface
  - System returns to normal operation

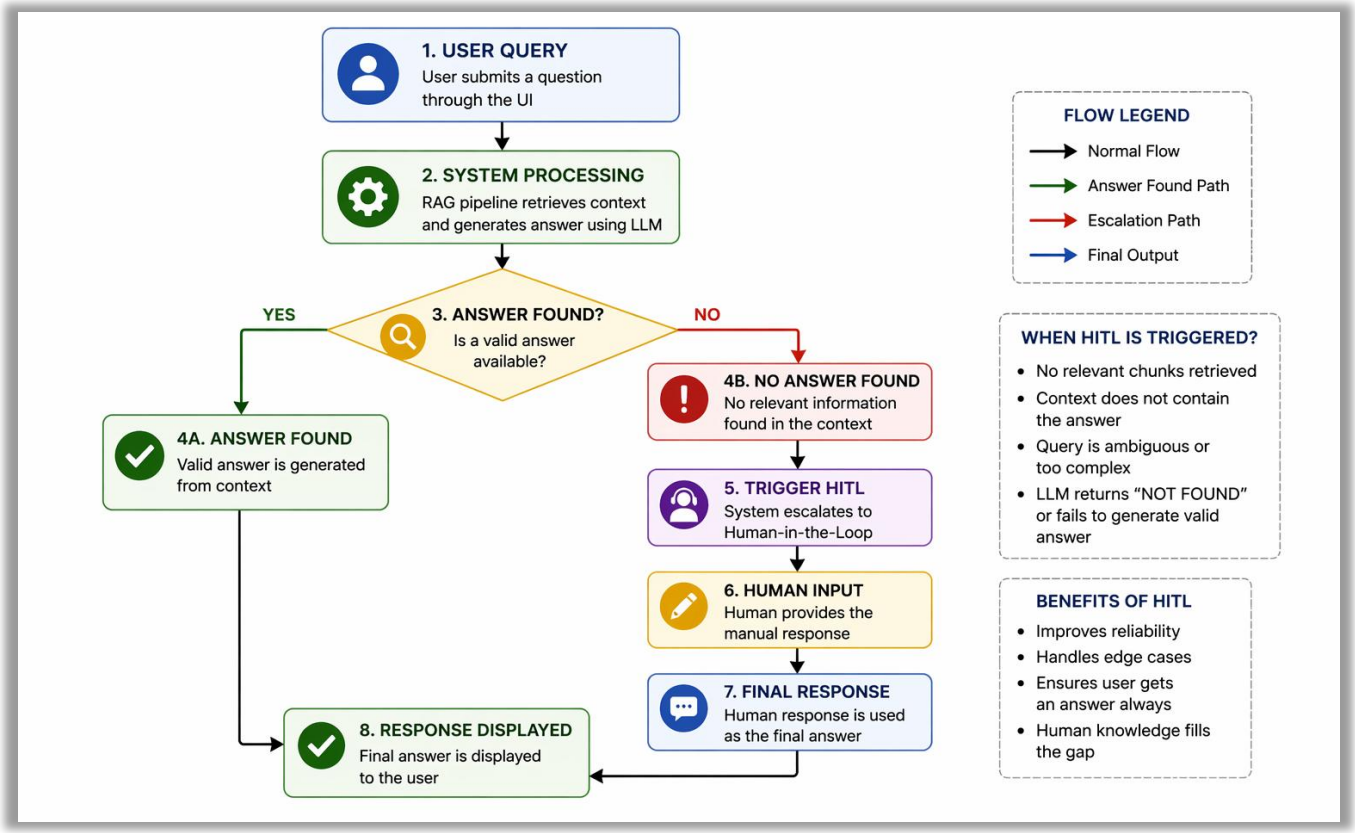

Fig: HITL Flow Diagram

6. API / Interface Design

Input Format

- User Query: string
- PDF File: uploaded document

Output Format

- Answer: string

  OR

- HITL Response

Interaction Flow

Upload PDF → Process → Ask Query → Retrieve → Answer / HITL

7. Error Handling

- **Missing Data**

  If no document is uploaded or processing fails, the system prevents query execution and prompts the user.

- **No Relevant Chunks Found**

  If retrieval returns no results, the system triggers HITL instead of generating incorrect answers.

- **LLM Failure**

  If the LLM API fails or returns an invalid response, the system falls back to HITL.